ESMERALDAS RESILIENTE 2026

DOCUMENTACIÓN TÉCNICA INTEGRAL

Arquitectura · Datos · API · Seguridad · IA · Despliegue · Operación

Hackathon 2026

Universidad Técnica Luis Vargas Torres de Esmeraldas

Integrantes:

Yeron Pool Cuero Montaño

Roberth Eladio Sanchez Quiñonez

Steven Enrique Burbano Chere

Adrián Andrés Sosa Vásquez

«Programar hoy para proteger vidas mañana»

Versión documental

Ecuador – Esmeraldas

Noviembre de 2026

Introducción

El fenómeno de El Niño constituye una amenaza natural que puede generar condiciones climáticas adversas, entre ellas lluvias intensas, inundaciones y afectaciones a la infraestructura, la movilidad y las actividades cotidianas de la población. En la ciudad y provincia de Esmeraldas, estos escenarios representan un desafío para la gestión del riesgo y la atención de emergencias, debido a la necesidad de disponer de información oportuna, organizada y geográficamente referenciada que facilite la toma de decisiones y la coordinación de las acciones de respuesta.

Ante esta problemática surge Esmeraldas Resiliente 2026, una propuesta tecnológica desarrollada en el marco del Hackathon 2026 de la Universidad Técnica Luis Vargas Torres de Esmeraldas (UTLVTE), orientada al diseño de una plataforma web colaborativa para apoyar la prevención, el monitoreo y la respuesta ante posibles emergencias relacionadas con el fenómeno de El Niño por lo cual esta iniciativa busca integrar información procedente de reportes ciudadanos, registros operativos, datos geográficos y servicios externos, con el propósito de proporcionar una visión territorial más organizada de las situaciones de riesgo.

La plataforma contempla herramientas para visualizar zonas vulnerables mediante mapas interactivos, registrar incidentes con ubicación geográfica, facilitar el envío de reportes incluso cuando no existe conexión a internet y ofrecer al Centro de Operaciones de Emergencia (COE) un tablero de control con indicadores y actualizaciones en tiempo real por lo tanto incorpora funcionalidades para consultar albergues, identificar posibles restricciones viales, organizar información relacionada con familias y asistencia humanitaria, e integrar mecanismos experimentales de analítica e inteligencia artificial que permitan apoyar la priorización de incidentes y el reconocimiento de patrones de recurrencia.

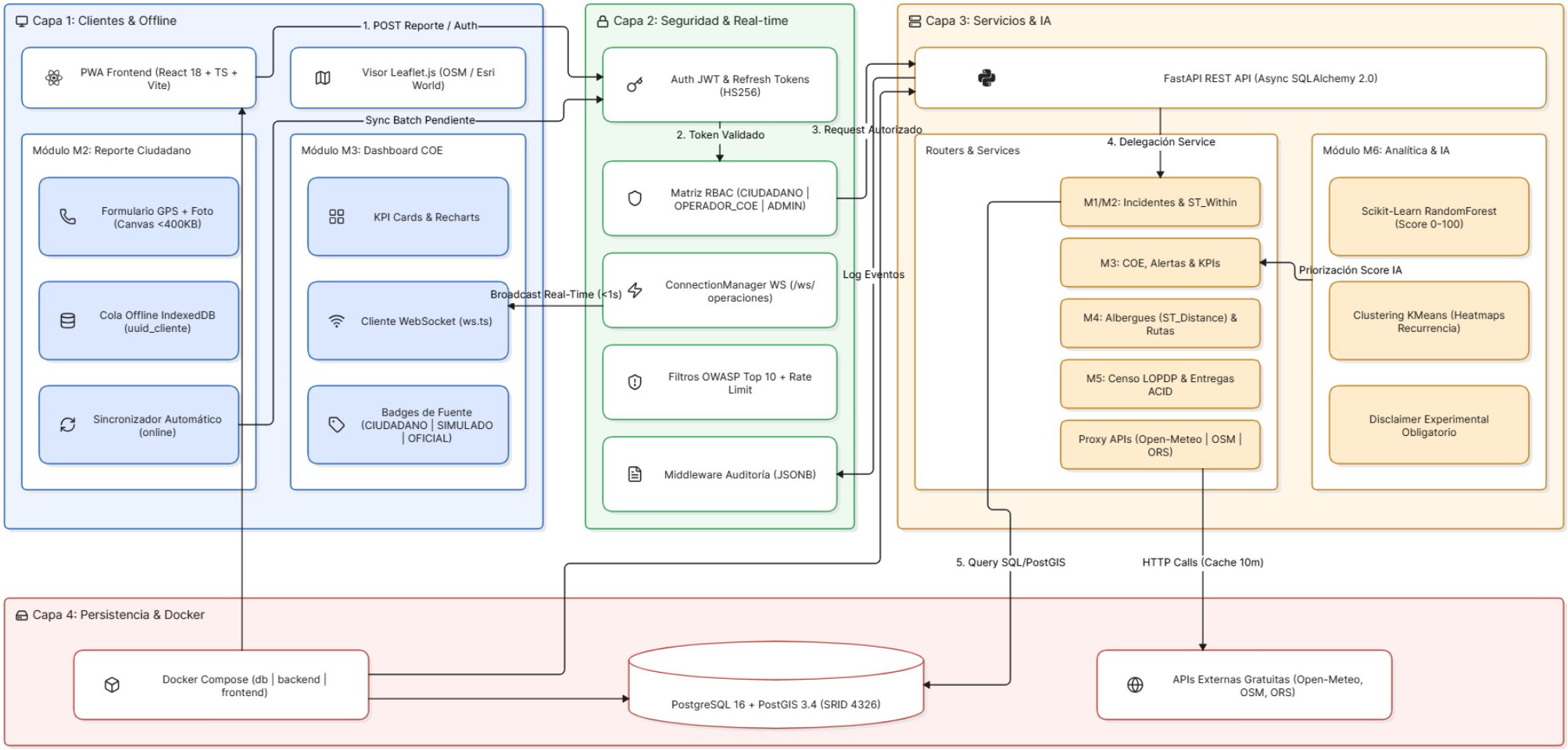
Desde el punto de vista tecnológico, la solución se plantea mediante una arquitectura organizada en capas que integra tecnologías de desarrollo web, servicios de backend, bases de datos geoespaciales, mecanismos de autenticación y autorización, comunicación en tiempo real y herramientas de análisis de datos mediante esta estructura busca facilitar la integración de sus componentes, mantener la coherencia de la información y ofrecer una solución escalable dentro del alcance de un prototipo académico.

La presente documentación tiene como finalidad describir de manera estructurada los fundamentos, la arquitectura, los componentes, el modelo de datos, los servicios, los mecanismos de seguridad, las integraciones, las funcionalidades y los procedimientos técnicos relacionados con Esmeraldas Resiliente 2026 y de esta manera se establece una referencia para comprender el funcionamiento previsto de la plataforma, orientar su uso y operación, facilitar su evaluación técnica y respaldar su presentación ante el jurado del Hackathon.

Es importante señalar que Esmeraldas Resiliente 2026 constituye una herramienta académica de apoyo y simulación y por lo tanto no reemplaza los sistemas oficiales de gestión de emergencias ni los canales institucionales de atención, ante una emergencia real la ciudadanía debe comunicarse con el ECU 911 y seguir las indicaciones de las autoridades competentes.

Diagrama General de Arquitectura

Esmeraldas Resiliente 2026 — Arquitectura & Flujo de Datos



1. Descripción del Proyecto

1.1 Identificación

Nombre: Esmeraldas Resiliente 2026.

Evento: Reto de Hackathon 2026, Universidad Técnica Luis Vargas Torres de Esmeraldas.

Lema: «Programar hoy para proteger vidas mañana».

Tipo de solución: aplicación web progresiva (PWA), adaptable a escritorio y dispositivos móviles.

1.2 Problema que aborda

Durante lluvias intensas e inundaciones, la información sobre incidentes, vías afectadas, personas que requieren evacuación, albergues y suministros puede encontrarse dispersa. La solución plantea reunir reportes ciudadanos georreferenciados y datos operativos en un entorno común para facilitar la visualización territorial y la coordinación del Centro de Operaciones de Emergencia (COE).

El contexto territorial de referencia incluye los antecedentes de inundación del río Teaone y del río Esmeraldas descritos en el Prompt Maestro. Las cifras y declaratorias allí recogidas deben conservar su fecha y fuente al reutilizarse; no deben presentarse como información en tiempo real sin verificación.

1.3 Objetivo general

Diseñar una plataforma web georreferenciada y colaborativa que permita registrar y visualizar incidentes, identificar zonas de riesgo, consultar albergues y servicios esenciales, organizar necesidades humanitarias y apoyar la priorización operativa ante escenarios de emergencia en Esmeraldas.

1.4 Objetivos específicos

1. Visualizar en un mapa las zonas de riesgo, incidentes, ríos, vías, albergues y servicios esenciales.
2. Facilitar el reporte ciudadano con ubicación GPS, fotografía opcional y almacenamiento temporal sin conexión.
3. Proporcionar al COE un tablero de incidentes, indicadores, alertas y actualizaciones en tiempo real.
4. Apoyar la consulta de albergues disponibles y la identificación de posibles restricciones viales.
5. Organizar el registro de familias, necesidades, inventario y entregas de asistencia con controles de integridad.
6. Incorporar analítica experimental para priorizar incidentes y visualizar patrones de recurrencia, sin sustituir el criterio humano ni los pronósticos oficiales.

1.5 Alcance funcional

El sistema se organiza en seis módulos:

- M1. GIS y mapa inteligente de riesgos: Visor cartográfico, capas GeoJSON, semáforo de riesgo y análisis espacial.
- M2. Reporte ciudadano offline-first: Formulario de incidentes, GPS, fotografía, cola local y sincronización idempotente.
- M3. COE y dashboard en tiempo real: KPIs, filtros, ciclo de vida de incidentes, alertas y cronología.

- M4. Evacuación y albergues: Ocupación, búsqueda por proximidad, servicios esenciales y advertencias por vías bloqueadas.
- M5. Familias y asistencia humanitaria: Censo, necesidades, inventario y entregas transaccionales.
- M6. Analítica e IA: Puntuación experimental de prioridad y agrupación de recurrencias.

La especificación clasifica M1, M2 y M3 como MUST; M4 y M5 como SHOULD; y M6 como COULD por lo tanto la documentación contempla las areas planteadas, pero la entrega real debe indicar qué módulos se implementaron y cuáles permanecen como alcance planificado.

1.6 Actores y permisos

Actor • Rol del sistema • Responsabilidades principales

Ciudadanía • `CIUDADANO` • Registrar reportes, consultar sus reportes y visualizar información pública

Operador COE • `OPERADOR_COE` • Revisar incidentes, cambiar estados, emitir alertas, gestionar albergues, familias y asistencia

Administrador • `ADMIN` • Gestionar usuarios, roles, estados de cuenta y auditoría administrativa

1.7 Principios de diseño

- Diferenciar claramente reportes ciudadanos, datos simulados y alertas oficiales.
- Mantener una arquitectura monolítica en capas, adecuada al tiempo y alcance del hackathon.
- Centralizar la lógica geoespacial en PostGIS.
- Dar prioridad a la operación offline del formulario ciudadano.
- Proteger datos personales y registrar operaciones sensibles.
- Mostrar de forma visible el carácter académico y experimental de la plataforma.

2. Arquitectura de software

2.1 Estilo arquitectónico

La solución adopta una arquitectura web monolítica organizada en cuatro capas y desplegada mediante Docker Compose mientras que el frontend SPA se comunica con una API REST y un canal WebSocket; la API concentra la lógica de negocio y accede a PostgreSQL/PostGIS mediante repositorios.

Esta decisión evita introducir complejidad innecesaria de microservicios en un prototipo de 24 a 72 horas y los módulos funcionales se mantienen separados dentro de una misma aplicación backend.

2.2 Capas

Capa 1. Clientes y operación offline

- PWA construida con React 18, TypeScript y Vite.
- Visor Leaflet con OpenStreetMap y Esri World Imagery.
- Formularios de reporte y panel COE.
- IndexedDB para almacenar reportes pendientes.
- Sincronización por lotes cuando se recupera la conectividad.

Capa 2. Seguridad y tiempo real

- Autenticación mediante JWT y refresh tokens.
- Autorización basada en roles: ciudadano, operador COE y administrador.
- WebSocket para difundir eventos operativos.
- Filtros OWASP, validación, limitación de peticiones y middleware de auditoría.

Capa 3. Servicios e inteligencia

- FastAPI como API REST asíncrona.
- Routers, servicios y repositorios.
- Servicios de incidentes, alertas, albergues, familias, inventario y KPIs.
- Proxy backend hacia APIs externas.
- Módulo analítico con scikit-learn.

Capa 4. Persistencia e infraestructura

- PostgreSQL 16 + PostGIS 3.4.
- Geometrías WGS84 con SRID 4326.
- Docker Compose para base de datos, backend y frontend.
- Datos de fuentes externas gratuitas y fuentes oficiales consultadas por operadores.

2.3 Diagrama lógico de componentes

```
flowchart TB
  subgraph C1["CAPA 1 · CLIENTES"]
    UI["PWA React 18 + TypeScript"]
    MAP["Leaflet · OSM / Esri"]
    OFF["IndexedDB · cola offline"]
  end
  subgraph C2["CAPA 2 · SEGURIDAD Y TIEMPO REAL"]
    AUTH["JWT · Refresh · RBAC"]
    WS["WebSocket /ws/operaciones"]
    AUD["Middleware de auditoría"]
  end
  subgraph C3["CAPA 3 · SERVICIOS E IA"]
    API["FastAPI REST"]
    SVC["Routers · Services · Repositories"]
```

```
  ML["Scikit-learn"]
  PROXY["Proxy de APIs externas"]
end
subgraph C4["CAPA 4 · DATOS E INFRAESTRUCTURA"]
  DB[("PostgreSQL 16 + PostGIS 3.4")]
  DOCKER["Docker Compose"]
  EXT["Open-Meteo · OSM · ORS"]
end
UI --> AUTH
UI <--> WS
OFF --> API
AUTH --> API
API --> SVC
SVC --> DB
API --> AUD
ML --> API
PROXY --> EXT
DOCKER --- DB
DOCKER --- API
DOCKER --- UI
```

2.4 Flujo de un reporte ciudadano

- La persona completa el formulario, adjunta opcionalmente una fotografía y obtiene coordenadas GPS o selecciona el punto manualmente.
- Si existe conexión, el cliente envía el reporte a `POST /api/incidentes`.
- Si no existe conexión o falla el envío, el reporte se conserva en IndexedDB con un `uuid_cliente`.
- Al recuperar la conectividad, el cliente envía el lote a `/api/incidentes/sincronizar`.
- FastAPI valida el token y el contenido con Pydantic.
- El servicio consulta PostGIS con `ST_Within` para asociar el punto a una zona de riesgo.
- El repositorio guarda el incidente en una transacción.
- El servicio emite `incidente_creado` por WebSocket.
- El mapa y el dashboard COE reciben el evento y actualizan la interfaz.

2.5 Organización interna del backend

```
backend/app/
├── main.py
├── core/             # configuración, seguridad y conexión a BD
├── models/           # modelos SQLAlchemy y GeoAlchemy2
├── schemas/          # DTO y validación Pydantic
├── repositories/     # acceso SQL/PostGIS
├── services/         # reglas de negocio y eventos
├── routers/          # endpoints REST
├── ws/               # ConnectionManager
├── ml/               # entrenamiento y scoring
└── seed.py            # usuarios de demostración
```

Regla de dependencias: router → service → repository → model. Los routers no deben contener consultas SQL ni reglas de negocio; los repositorios son responsables del acceso a datos; los servicios coordinan reglas y eventos.

2.6 Decisiones arquitectónicas (ADR)

ADR • Decisión • Alternativa descartada • Justificación

ADR-01 • Monolito en capas con Docker Compose • Microservicios • Menor complejidad y fallos de integración en un hackathon

ADR-02 • FastAPI + SQLAlchemy async • Backend separado para IA • Unifica Python, API e IA y ofrece validación tipada

ADR-03 • WebSocket nativo • Socket.IO/MQTT • Suficiente para difusión en tiempo real

ADR-04 • IndexedDB + UUID • Solo Background Sync • Control de cola y reintentos idempotentes

ADR-05 • PostGIS • Cálculo espacial en navegador • Consultas espaciales centralizadas y reproducibles

ADR-06 • Proxy backend de APIs externas • Llamadas directas desde frontend • Control de caché, CORS, límites y secretos

2.7 Reglas geoespaciales

- Todas las geometrías se almacenan con SRID 4326.
- Las respuestas geográficas se serializan como GeoJSON conforme a RFC 7946.
- Las distancias se calculan sobre `geometry::geography` y se expresan en metros.
- `ST_Within` determina si un punto se encuentra dentro de un polígono.
- `ST_Distance` ordena albergues por distancia geodésica.
- `ST_Buffer` y `ST_Intersects` permiten detectar proximidad a vías interrumpidas.
- Se deben validar geometrías, coordenadas y orden longitud/latitud antes de persistir.

2.8 Lectura del diagrama

El diagrama visual presenta el frontend PWA y visor Leaflet en la capa 1; JWT, matriz RBAC, WebSocket, filtros OWASP y auditoría en la capa 2; FastAPI, routers de M1–M5, proxy y analítica IA en la capa 3; y Docker Compose, PostgreSQL/PostGIS y APIs externas en la capa 4. Esta lectura es la referencia visual de la arquitectura.

